DEVIXO SOLUTIONS

PROJECT REPORT

AppointEase – Appointment Booking System

Submitted By: Abubakar Siddique
Organization: Devixo Solutions
Task: Task 04 – Full-Stack Application
Project Domain: Appointment Booking System
Technologies: React, Node.js, Express.js, MongoDB

GitHub Repository:
https://github.com/abubakar136/AppointEase

---

1. Introduction

AppointEase is a full-stack appointment booking web application developed to help users manage their appointments through a simple and user-friendly interface.

The application provides user registration, login, appointment management, and a dashboard for viewing and organizing appointment records.

The project was developed using React for the frontend, Node.js and Express.js for the backend, and MongoDB for database storage.

2. Project Objectives

The main objectives of AppointEase are:

- To develop a responsive appointment booking website.
- To allow users to register and log in securely.
- To allow users to create, view, edit, and delete appointments.
- To provide a dashboard for managing appointments.
- To store user and appointment data in MongoDB.
- To provide search and filtering functionality.
- To prepare the application for online deployment.

3. Problem Statement

Managing appointments manually can make it difficult to organize records, find upcoming bookings, and update appointment details.

AppointEase solves this problem by providing a centralized web application where users can manage their appointments, check their status, and update information easily.

4. Technologies Used

1. React: Used to develop the frontend and user interface.

2. Vite: Used as the frontend development and build tool.

3. Node.js: Used to run the backend JavaScript application.

4. Express.js: Used to create backend APIs and handle HTTP requests.

5. MongoDB Atlas: Used to store user and appointment data.

6. Mongoose: Used to define database models and communicate with MongoDB.

7. JWT: Used for token-based user authentication.

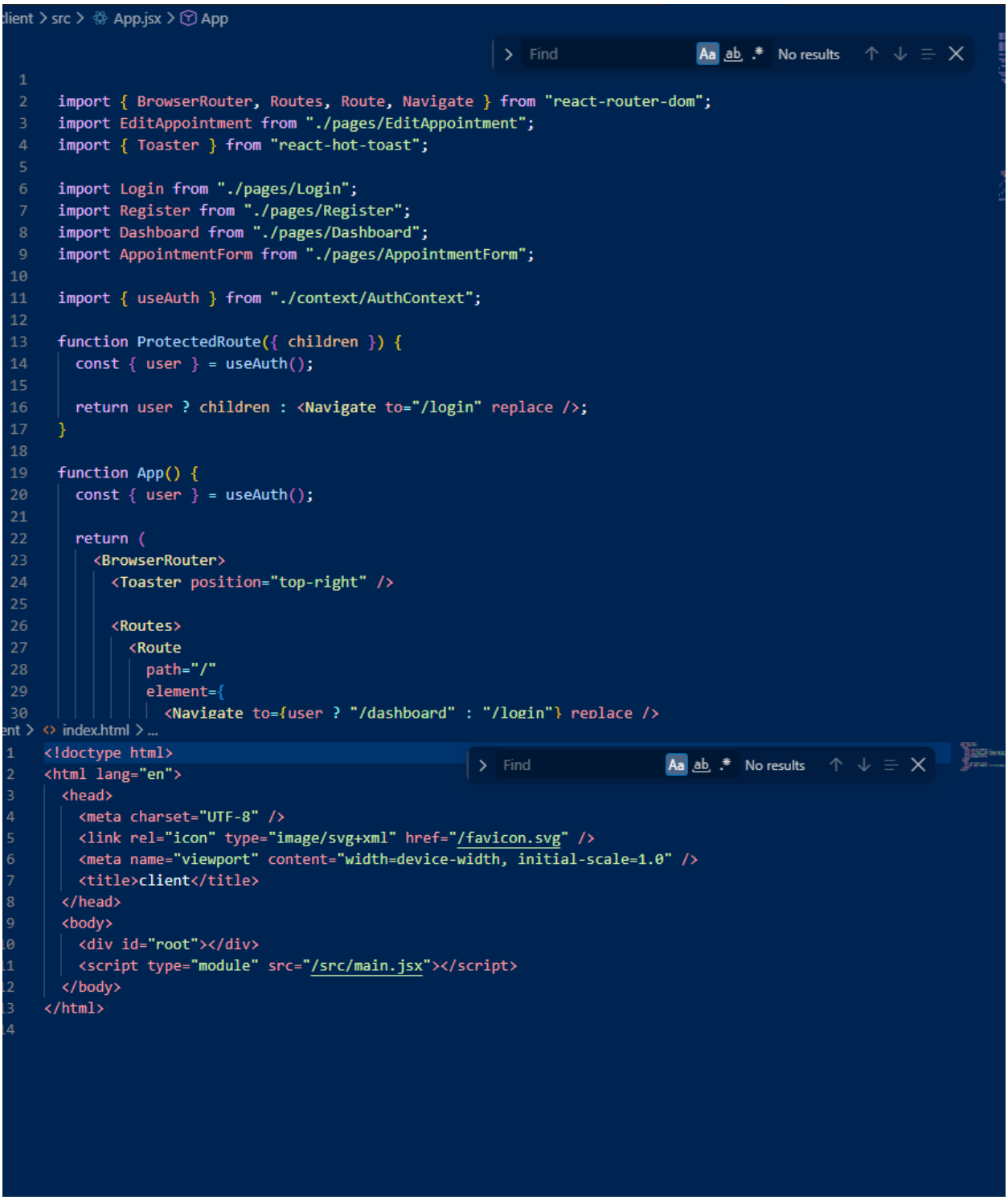
8. bcryptjs: Used to hash passwords before storing them.

9. Axios: Used to send API requests from the frontend.

10. Git and GitHub: Used for version control and source-code hosting.

11. Vercel: Used as the deployment platform for the project.

Codes


lient > src > App.jsx > App
Find
Aa ab .* No results
1
2 import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
3 import EditAppointment from "./pages/EditAppointment";
4 import { Toaster } from "react-hot-toast";
5
6 import Login from "./pages/Login";
7 import Register from "./pages/Register";
8 import Dashboard from "./pages/Dashboard";
9 import AppointmentForm from "./pages/AppointmentForm";
10
11 import { useAuth } from "./context/AuthContext";
12
13 function ProtectedRoute({ children }) {
14 const { user } = useAuth();
15
16 return user ? children : <Navigate to="/login" replace />;
17 }
18
19 function App() {
20 const { user } = useAuth();
21
22 return (
23 <BrowserRouter>
24 <Toaster position="top-right" />
25
26 <Routes>
27 <Route
28 path="/"
29 element={
30 <Navigate to={user ? "/dashboard" : "/login"} replace />
ent > index.html > ...
Find
Aa ab .* No results
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7 <title>client</title>
8 </head>
9 <body>
10
11 <script type="module" src="/src/main.jsx"></script>
12 </body>
13 </html>
14

client > src > # index.css > { } @media (max-width: 650px)

Find Aa ab .* No results

```css

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', sans-serif;
  background: #f5f7fb;
  color: #1e293b;
}

button,
input,
textarea {
  font-family: inherit;
}

button {
  cursor: pointer;
  border: none;
}

a {
  text-decoration: none;
  color: #4f46e5;
}
```

5. Main Features

5.1 User Authentication

The application provides registration and login functionality.

Users can create accounts and log in using their credentials. Passwords are hashed using bcryptjs, and JSON Web Tokens (JWT) are used to authenticate protected API requests.

5.2 Appointment Management

Users can:

- Create new appointments.
- View appointment records.
- Edit appointment details.
- Delete appointments.
- Change appointment status.

Each appointment contains:

- Title
- Description
- Date
- Start Time
- End Time
- Status

The application supports four appointment statuses:

- Pending
- Confirmed
- Cancelled
- Completed

5.3 Dashboard

The dashboard displays important appointment information.

It includes:

- Total appointments
- Pending appointments
- Confirmed appointments
- Cancelled appointments
- Completed appointments
- Recent appointments

5.4 Search and Filtering

Users can search appointments by title and filter them according to their status.

This makes it easier to find and manage specific appointments.

5.5 Responsive Interface

The frontend is designed to work on different screen sizes, including desktop and smaller-screen devices.

6. System Architecture

AppointEase follows a frontend, backend, and database architecture.

Frontend: React is used to display pages, forms, dashboard information, and appointment records.

Backend: Node.js and Express.js handle API requests, authentication, validation, and appointment operations.

Database: MongoDB Atlas stores user accounts and appointment records.

The frontend communicates with the backend through API requests. The backend processes the requests and communicates with the database.

7. Database Design

The project uses MongoDB Atlas as its database.

It contains two main models:

7.1 User Model

The User model stores account information, including:

- Name
- Email
- Hashed Password

7.2 Appointment Model

The Appointment model stores:

- User reference
- Appointment title
- Description
- Date
- Start time
- End time
- Status

Each appointment is associated with a user. The backend checks the authenticated user before accessing or modifying appointment records.

8. API Implementation

The backend provides the following API endpoints:

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Log in a user
GET	/api/appointments	Get appointments
POST	/api/appointments	Create an appointment
PUT	/api/appointments/	Update an appointment
DELETE	/api/appointments/	Delete an appointment
GET	/api/health	Check backend health

Protected appointment routes require a valid authentication token.

9. Testing and Validation

The application was tested during development using the local frontend, backend, and API requests.

The following areas were tested:

- User registration.

- User login.
- Authentication using JWT.
- Appointment creation.
- Appointment retrieval.
- Appointment updating.
- Appointment deletion.
- Appointment status changes.
- Search and filtering.
- Appointment time validation.
- Frontend production build.

The frontend production build completed successfully using the Vite build command.

The live deployment should also be tested to confirm that registration, login, database connection, and appointment management work correctly.

10. Challenges and Solutions

During development, some challenges were faced.

10.1 MongoDB Atlas Connection

Sometimes the backend could not connect to MongoDB Atlas because the connecting IP address was not allowed.

Solution: The Atlas Network Access settings must allow the environment connecting to the database.

10.2 Local and Production Configuration

The application needed different configurations for local development and Vercel deployment.

Solution: The Express application was prepared to support local execution and a Vercel serverless entry point.

10.3 Authentication

Protected API routes needed to verify the user's identity.

Solution: JWT authentication middleware was implemented to validate tokens.

10.4 Appointment Validation

Appointment times needed validation to prevent invalid time ranges.

Solution: Validation was added to ensure that the end time is later than the start time.

11. Project Structure

The project is organized into separate frontend and backend folders.

AppointEase/

- api/
  - index.mjs
- client/
  - React frontend
  - Components
  - Pages
  - API services
- server/
  - config/
  - controllers/
  - models/
  - routes/
  - server.js
- vercel.json
- README.md
- .gitignore

12. Running the Project Locally

Backend

Open a terminal in the server folder and run:

npm install

npm run dev

Frontend

Open another terminal in the client folder and run:

npm install

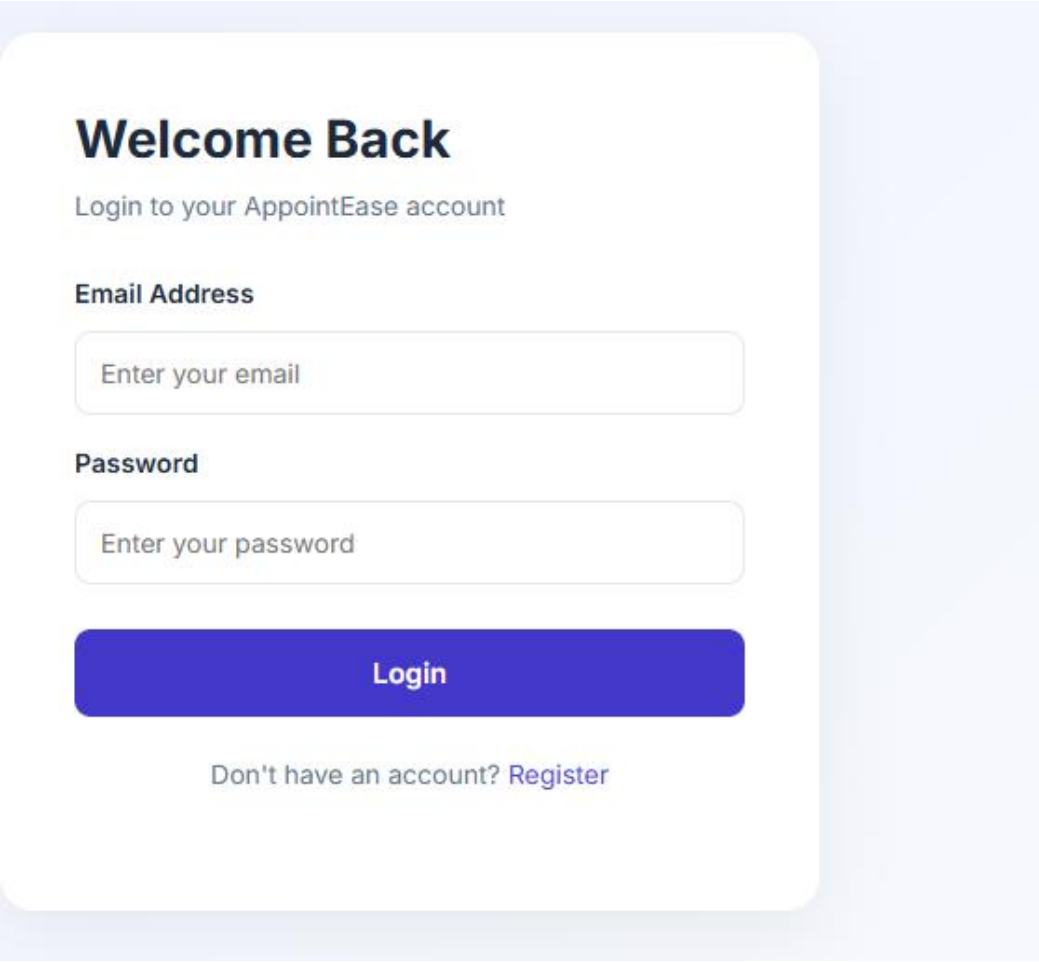
npm run dev

The backend requires environment variables for the MongoDB connection string and JWT secret.

These private values should be stored in environment files and should not be uploaded to GitHub.

13. Screenshots

Screenshot 1: Registration and Login Page



Create Account

Join AppointEase today

Full Name

Enter your name

Email Address

Enter your email

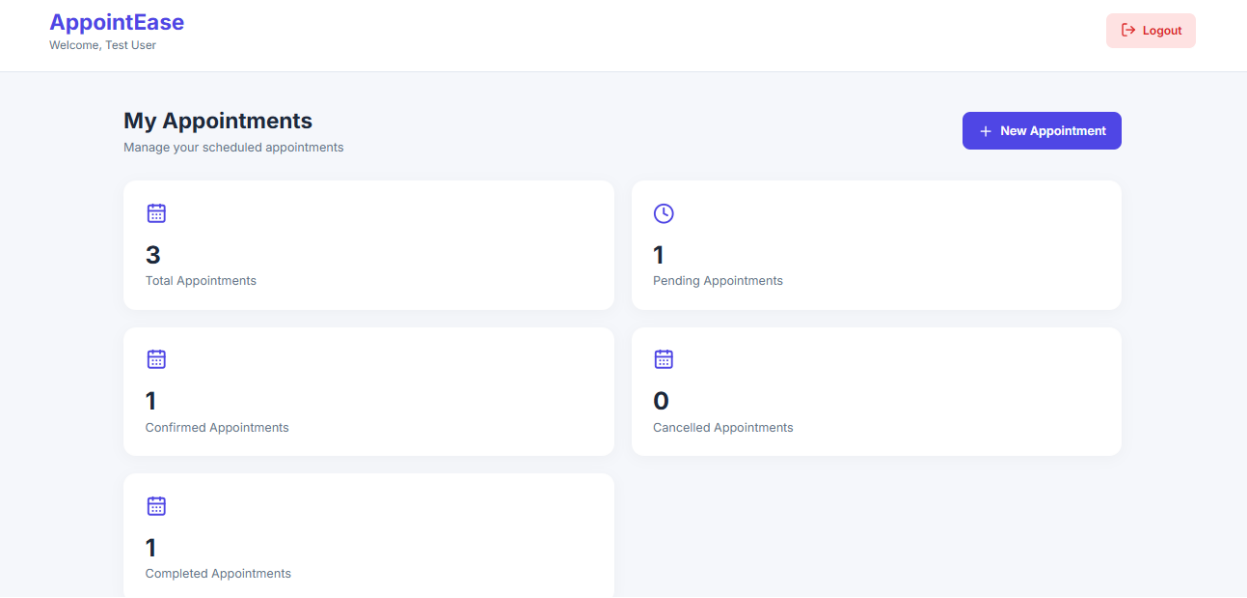
Password

Create a password (minimum 8 characters)

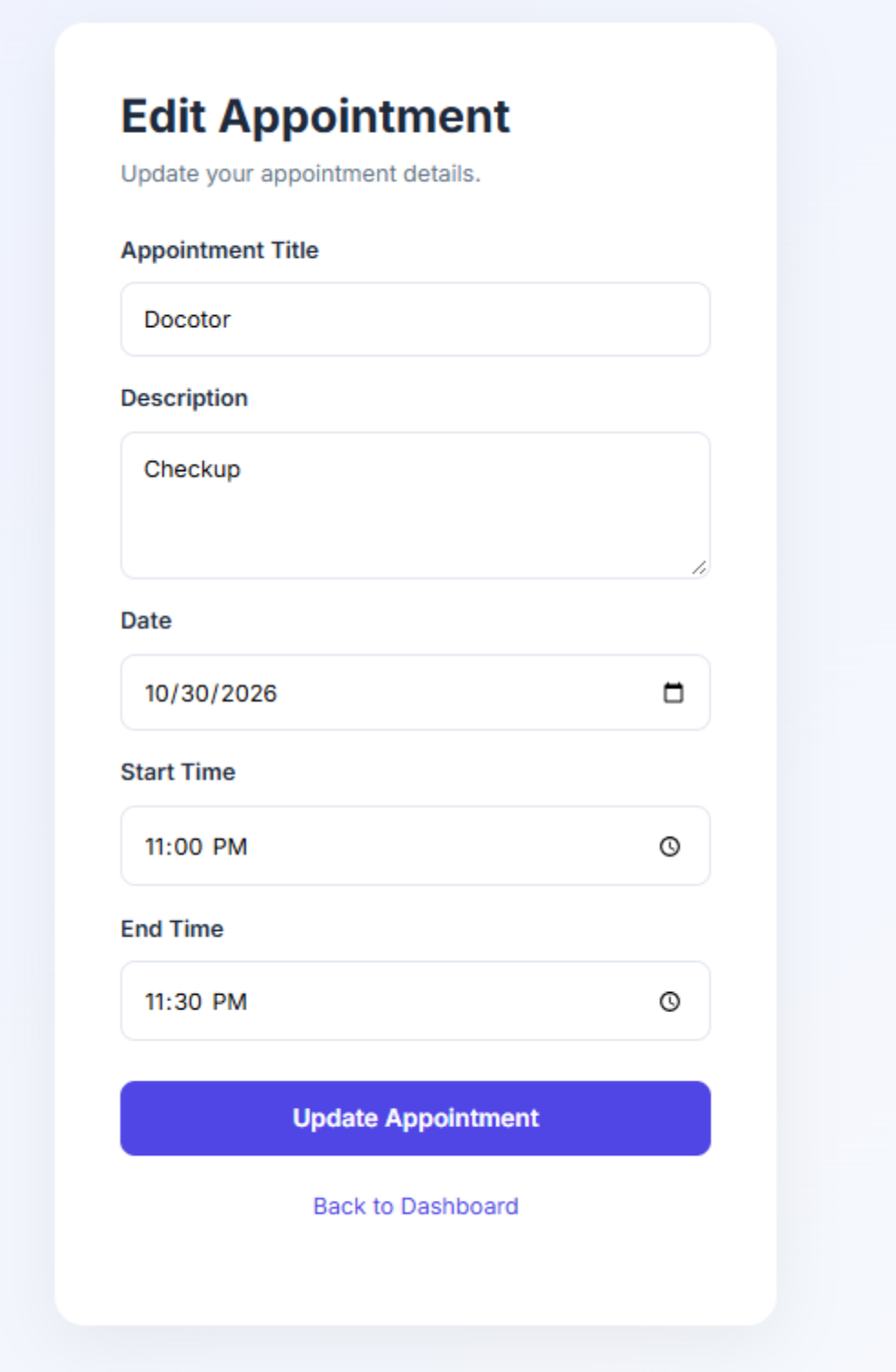
Register

Already have an account? Login

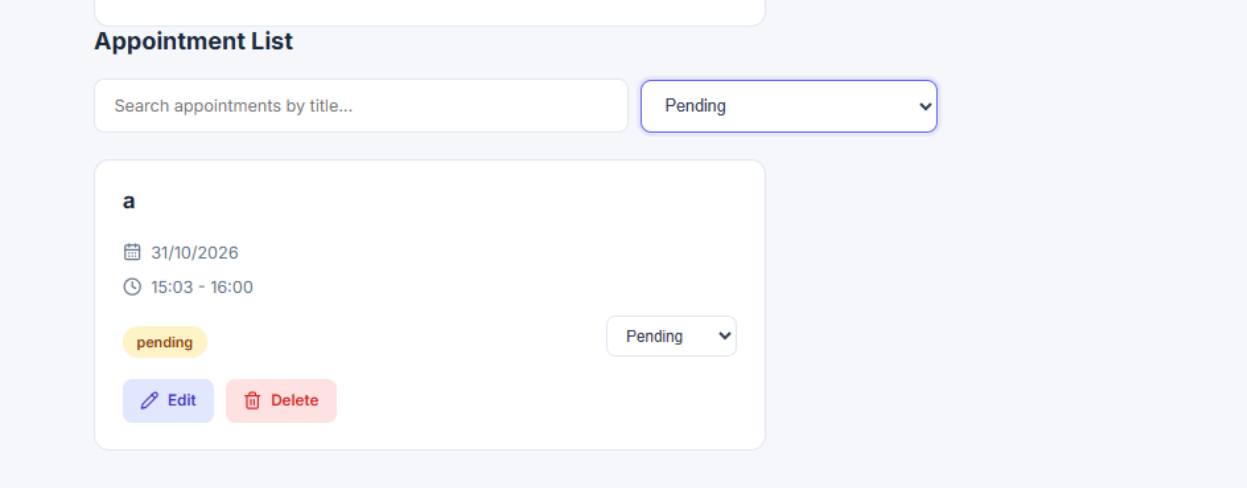
Screenshot 2: AppointEase Dashboard



Screenshot 3: Create or Edit Appointment



Screenshot 4: Appointment List and Status Filters



14. Conclusion

AppointEase is a full-stack appointment booking application developed using modern web technologies.

It provides user authentication, appointment management, dashboard statistics, search and filtering, data validation, and MongoDB database integration.

This project helped demonstrate practical knowledge of frontend development, backend development, database management, API integration, authentication, and deployment preparation.

The project source code is available on GitHub, and the application has been prepared for Vercel deployment.

15. Project Links

GitHub Repository:
https://github.com/abubakar136/AppointEase

Live Website:
https://appoint-ease-7r1c-git-main-abubakar136.vercel.app

Submitted By: Abubakar Siddique